

Is time all that matters? Exploring Context-Aware-Recommendation-Systems to exploit novelty and diversity

Team Members:

Leonardo Eshun

Meron Gedrago

Mobasserul Haque

Diego Rodriguez

Abstract

Personalized recommendations have become pivotal to user engagement on streaming and online shopping platforms, where accurate suggestions directly influence customers' spending, viewing time, and subscriber retention. This study evaluates and enhances the Short-Term and Long-Term preference Integrated Recommender system (SLi-Rec) model across four key dimensions: generalization, domain transfer, novelty recommendation, and contextual item modeling. To investigate these capabilities in depth, we conduct four experiments driven by practical deployment considerations. First, the model's robustness is tested under different availability of data using different sampling strategies, demonstrating consistent performance as dataset representativeness increases. Second, domain transfer experiments reveal that SLi-Rec performs poorly when applied to unseen categories without retraining, underscoring its sensitivity to domain-specific user behavior and data richness. Third, we introduce a temperature hyperparameter to control the diversity of recommendations, showing its utility in adjusting between focused and exploratory outputs with varying AUC performance depending on the specific need. Finally, we integrate a graph-based module using Light Graph Convolution Network (LightGCN) to enrich item context, significantly improving ranking metrics, particularly in cold-start scenarios, improving AUC from 0.83 to 0.88. These explorations and adjustments establish a more adaptive and context-aware SLi-Rec architecture for personalized recommendation tasks.

Introduction

Recommendations are everywhere and highly important. They impact both the consumer experience and the performance of companies. Take movies and TV, for example, for content providers and studios, surfacing the right title or product at the right moment directly translates into watch time, subscription retention, and revenue. Yet the sheer scale of modern catalogs—often exceeding 100,000 titles—makes manual curation infeasible. Therefore, stakeholders need recommender systems that capture evolving viewer tastes and adapt quickly as new content is released. Our work addresses these needs by examining SLi-Rec (Figure 1), a state-of-the-art sequential model used by Microsoft, through the lens of real-world constraints such as data sparsity and cold-start items, and by proposing targeted extensions that bolster its effectiveness in production settings.

Short-term and Long-term preference Integrated RECommender system (SLi-Rec)

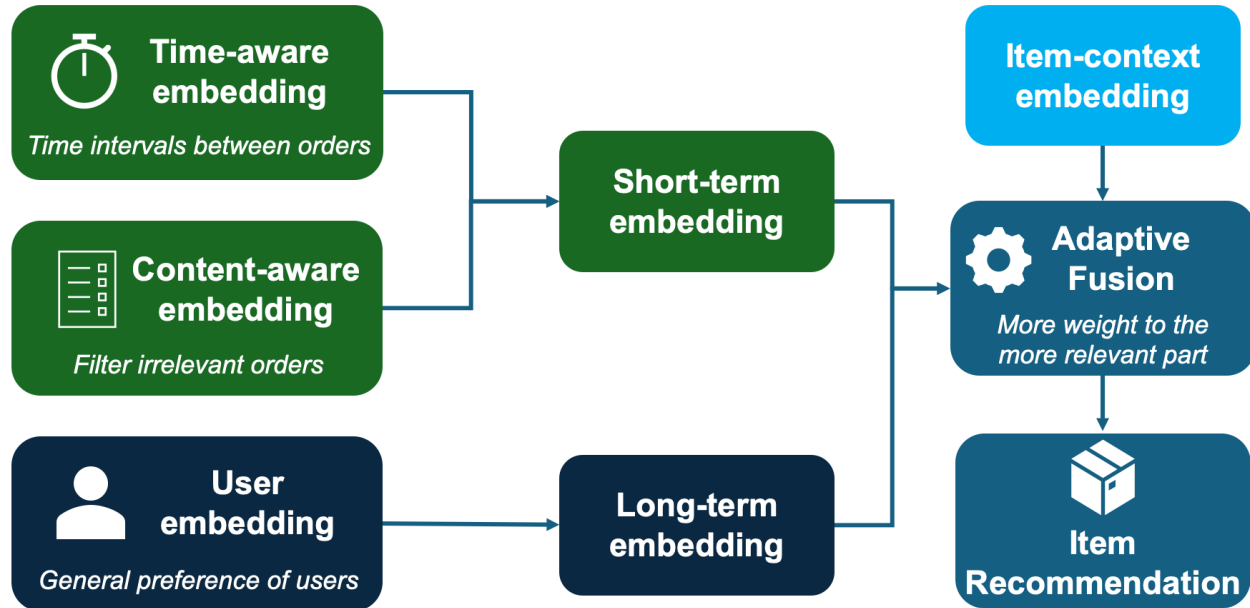


Figure 1: Model Diagram of SLi-Rec

The SLi-Rec model looks at both what users like in general and what they are interested in right now. It captures recent behaviors using time and content information, like how long it's been since a purchase and whether the items are relevant. At the same time, it learns a user's long-term preferences from their overall activity. These two types of preferences are then combined in a smart way that gives more importance to whichever is more useful at the moment, helping the system make better recommendations.

Specifically, we will conduct the following experiments to test SLi-Rec performance and flexibility: First, we will test its generalization performance by subsampling by attributes, like date, user, or at random. Second, we will evaluate its out-of-the-box transfer capabilities by training the model on 16 different categories and evaluating the performance across the groups. Third, we will evaluate how a temperature hyperparameter can affect the recommendations made by the system, trading some performance for novel recommendations. Fourth, we will enrich the item-related layer of the model with a LightGCN to increase performance beyond time dynamics.

Background

The evolution of recommender systems has been characterized by a shift from static preference models to dynamic, behavior-aware frameworks. Early approaches, such as Asymmetric-Singular Value decomposition (SVD) (Koren et al., 2009) and Funk-SVD (Simon, 2006), demonstrated that latent factor models could effectively capture user preferences. However, their focus on static data limited their ability to model the evolving nature of user interests and the sequential dynamics of interactions.

To address these limitations, subsequent research emphasized the incorporation of temporal dynamics into recommendation models. Sequential recommendation methodologies emerged using Markov chains (Koren, 2010) and recurrent neural networks (Johnson et al., 2017; Hidasi et al., 2018; Wang et al., 2018) to explicitly capture the order and timing of user interactions. Further refinements, including attention mechanisms (Liu et al., 2016; Zhu et al., 2017), have improved the modeling of long-term preferences alongside rapidly changing short-term behaviors.

In this context, Yu et al. (2019) introduced SLi-Rec, a deep learning model that integrates temporal and contextual features via attention mechanisms. A final attention layer mixes three components. First, the model incorporates Asymmetric-SVD for modeling long-term static preferences. Second, an enhanced Long Short-Term Memory (LSTM) network that uses a time-aware controller through both the cell state and output gate to capture short-term preferences. Third, SLi-Rec uses contextual embeddings, such as item and category information, to dynamically determine the relevance of short- or long-term signals based on the current context, as well as the third source of information for the model decision. This architecture enables greater flexibility and performance, making SLi-Rec well-suited for industrial applications across diverse sectors. However, despite its strengths, SLi-Rec, like other sequential models, remains constrained by its reliance on historical user-item interactions. In particular, it struggles in cold-start scenarios, where new or out-of-distribution (OOD) items lack temporal context.

Recent research has explored graph-based methodologies, which offer a promising alternative to purely sequential models by focusing on item similarity rather than time dynamics (Wang et al 2023, Zhang et al 2021). GNNs, such as those introduced by Kipf and Welling (2017), leverage the relational structure between users and items by modeling them as nodes in a graph with edges representing interactions. This approach allows GNNs to aggregate information from local neighborhoods and capture both low- and high-order connectivity patterns, which are often overlooked by sequence-only architectures. These capabilities make GNNs particularly well-suited to handling sparse data and recommending unseen items. This feature can be incorporated into the SLi-rec model to make it aware of item metadata and diminish the over-reliance on historical context.

Data

The SLi-Rec model in the original paper uses three Amazon subcategories (Electronics, Movies & TV, and CDs & Vinyl) and a proprietary industrial dataset (MSN native ads). The Amazon data includes product info, ratings, and user/item metadata, while the industrial dataset uses browsing and ad interaction data. Both are split into training, validation, and test sets.

The previously mentioned training datasets were collected and accessed before 2016. However, for all our experiments, we are using Amazon datasets collected in 2023 (Hou et al., 2024), spanning from May 1996 to September 2023. Three of our experiments will use the Movies and TV dataset, which includes 6.5 million users providing 17.3 million ratings across 747.8 thousand items. This dataset is the current standard for evaluating recommendation systems. This allows us to evaluate how the SLi-Rec model performs with richer and more recent data. In addition to the Movies and TV dataset, we explore how the SLi-Rec model performs on 15 Amazon categories, with a varying number of users, items, and reviews as shown in the figure below.

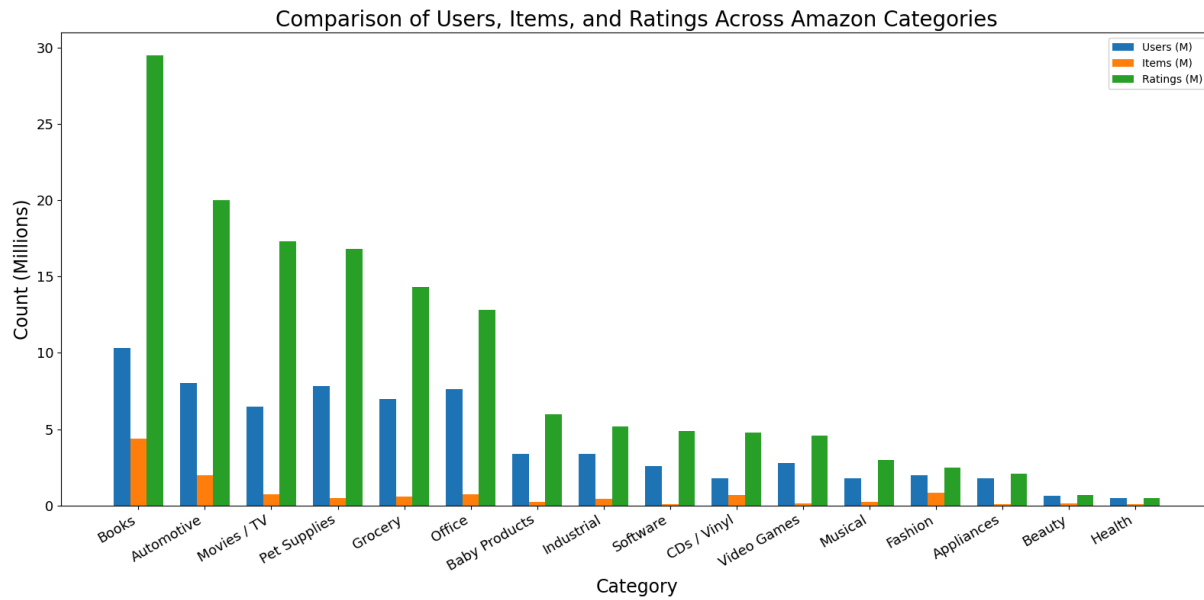


Figure 2: Distribution of Users, Items, and Ratings across different Amazon Categories used in this project

In order to correct the lack of negative instances in the dataset, the standard approach in recommendation systems is followed, known as negative sampling. This approach means that for each positive click, 4 negative instances are generated based on popularity sampling. This is repeated on the validation set and increased to 9 negative samples on the test set, aligning with the recommendations on the SLi-rec model (Yu et al, 2019).

Experiments

All four experiments are guided by one overarching question: how resilient is SLi-Rec when confronted with the key operational hurdles that production-grade recommenders inevitably face—data sparsity, domain shift, demand for controlled diversity, and missing contextual signals? By stress-testing the model along these axes, we obtain a comprehensive view of its strengths and the precise conditions under which targeted enhancements deliver the most tangible gains. These are the specific experiments:

- **Generalization performance:** This experiment aims to measure the robustness of the model under different handicaps, while using different sizes of training, validation, and testing on a 0.6/0.15/0.25 ratio. To accomplish this, the SLi-Rec model is evaluated under different data scenarios, exposing it to a quarter, half, three-quarters, and the totality of the Movies and TV dataset. Two subsets are used, one where observations are selected randomly from the full dataset, and where a fraction of the users are tracked. The performance was measured with the AUC score, and the time taken to train the model was recorded.
- **Domain transfer capabilities:** We tested it on new categories and also retrained and evaluated it on various category types to understand how the model adapts to new domains. First, Pre-trained Evaluation by testing a pre-trained SLi-Rec model (trained on Movies and TV,

Groceries, or Automotive category) on a different category that it was not trained on. Second, train and test the model separately on 14 new categories.

- **Increase diversity in recommendations:** This experiment aims to introduce randomness in the recommendations through a stochastic approach. To achieve this, we add a hyperparameter on the final layer before the computation of the probabilities to control the uniformity of the probability distribution by user recommendation, this is equivalent to a temperature hyperparameter on a natural language model. In practice, a value closer to 0 will lead to deterministic models, and a very high value will lead to a random model. This should be ideal for sequence recommendations trained on negative instances, as the model could pick among popular options that align with the time interactions of the user, especially when there is a limited set of options available to be displayed to the user.
- **Context-Aware SLi-Rec, Fusing SLi-Rec and Graph-Based Item Relationships:** This experiment aims to enhance the SLi-Rec recommendation model by making it context-aware by integrating item-item relational signals. This will let the model capture embedding distances between items, something that the baseline SLi-Rec was not doing before. We hypothesize that incorporating graph-based item context will improve the model's ability to generalize, particularly in sparse or cold-start scenarios where temporal patterns alone may be insufficient. Specifically, we extend SLi-Rec by integrating LightGCN, a simplified Graph Convolutional Network that learns item embeddings from a user-item graph during and is added during the adaptive fusion stage of SLi-Rec. To achieve this, the LightGCN goes through the following process. First, LightGCN is trained on the Amazon Movies and TV dataset to learn item-to-item relations. Second, this new graph embedding replaces the original item embedding of the fusion stage. Finally, the process follows the standard approach detailed on the SLi-rec model Figure 1 to produce a recommendation. Figure 3, summarizes the process.

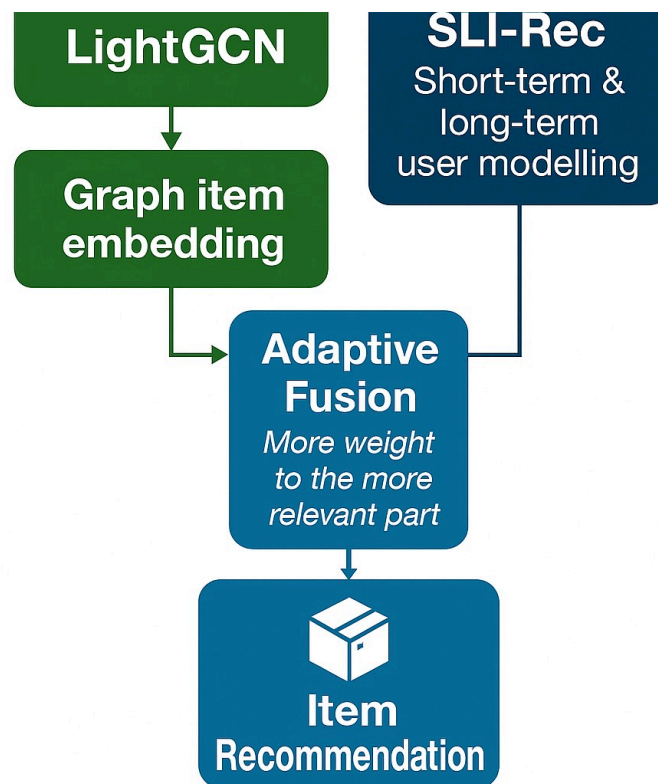


Figure 3: Architecture of the SLI_RECWithGCN model.

The resulting model, referred to as SLI_RECWithGCN, jointly reasons over both session-based behavior and graph-based item relationships, aiming to deliver more accurate and robust recommendations. We train and evaluate SLI_RECWithGCN on 10% of the Amazon Movies and TV dataset and compare its performance against the baseline SLi-Rec model. Evaluation metrics include AUC for comparison purposes with the baseline model, and MRR, NDCG@k, to evaluate top-k (the number of relevant items to recommend) metrics as they are relevant in restricted recommendation settings. Detailed definitions of these metrics are provided in Appendix B.3.

Results

Generalization performance:

Figure 4 shows the model performance across sampling. The model shows a similar performance of about 0.88 AUC score across sample sizes and types. This situation can be explained by the capacity of the model to learn and generalize regardless of the sample size, suggesting that data quality must be more important than sample size.

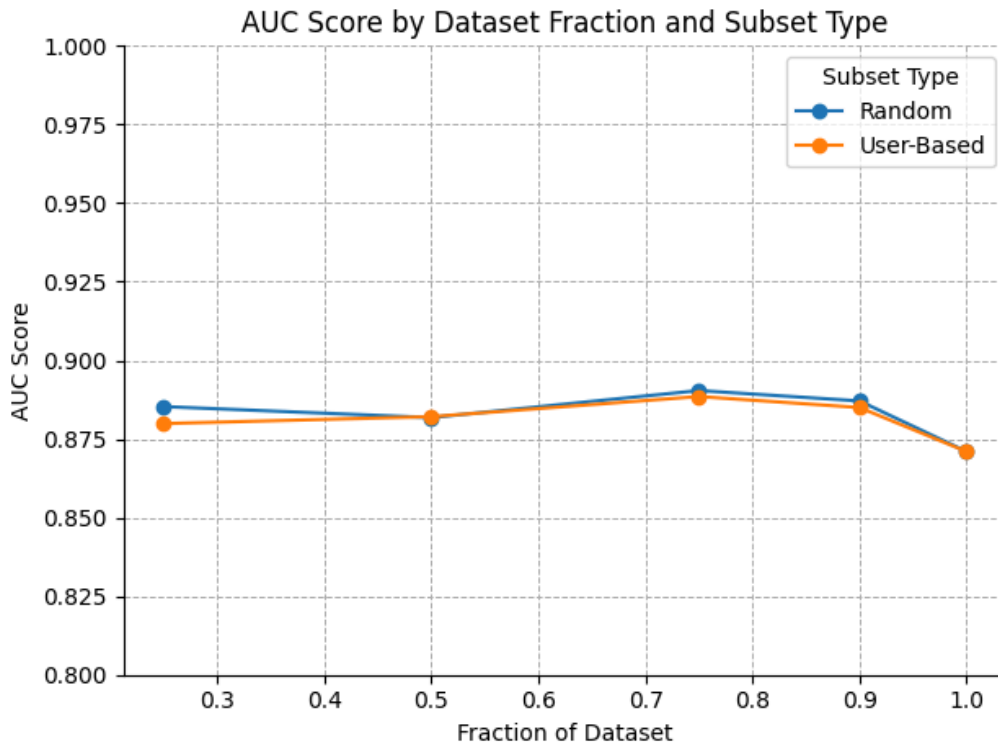


Figure 4: Model performance (AUC) across different dataset fractions.

However, Figure 5 shows that the training time increases linearly with the fraction of the dataset used. This is expected as the model increasingly gets more data to learn from.

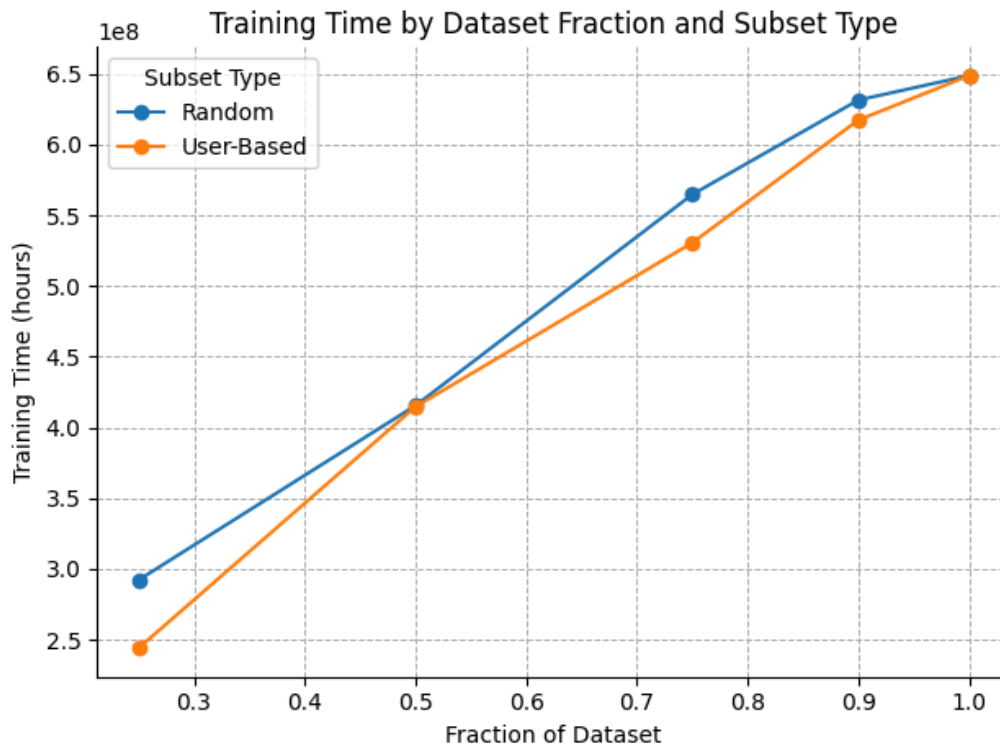


Figure 5: Model training time across different dataset fractions

Overall, the experiment suggests that the model is capable of generalizing well under different circumstances, including low and large training dataset sizes, as long as the dataset is representative of the underlying population.

Domain transfer capabilities:

When the pre-trained model on movies and TV is tested on new categories, the results show the model does not perform well, as illustrated in the figure below. This outcome is expected because the SLi-Rec model is trained on a user's review history within a specific category to predict their next interaction. When the model is applied to an entirely different dataset, it often encounters users who are not present in the training data and products that are completely unrelated. These differences limit the model's ability to generate meaningful recommendations without a new training session.

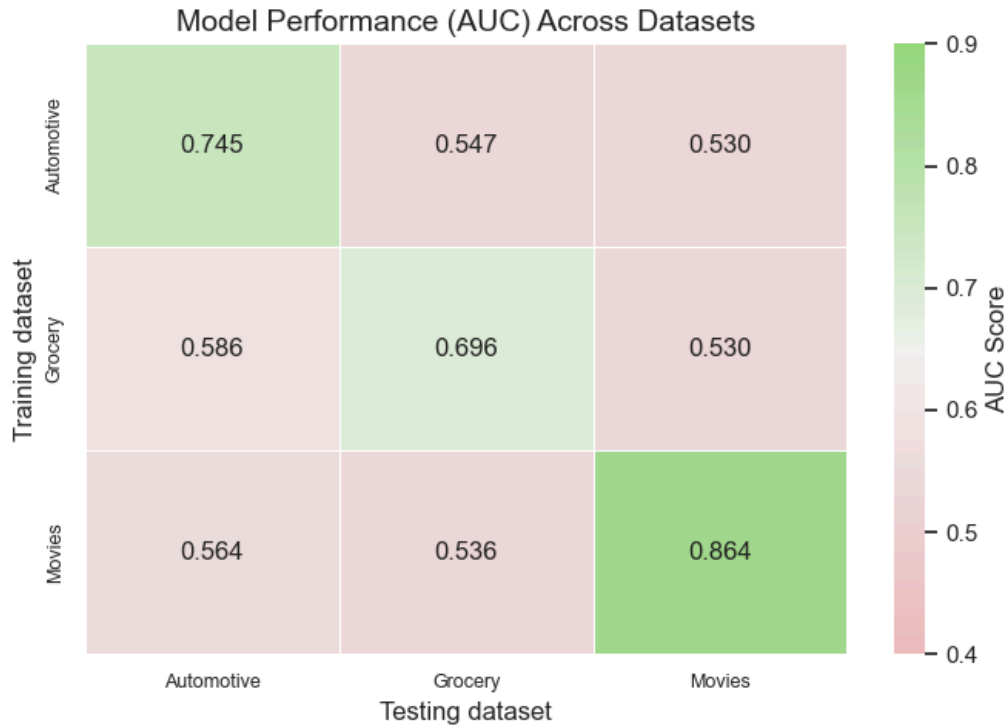


Figure 6. Model performance (AUC) across different training and testing dataset categories

However, a high AUC heterogeneity across the same training and testing was not expected. The AUC in the initial experiment ranged from 0.52 (health) to 0.86 (movies). One possible explanation is the level of interactions between items and users in each dataset. The literature confirms this, as movie datasets are often used as benchmarks for recommendation systems because of the abundant interactions between users and items (Harper & Konstan, 2015). This density or "richness" of a dataset can be measured (Dooms et al., 2016) as done in the formula below. As more users leave ratings on more products, the data density approaches 1.

$$\text{Data Density} = \frac{\text{Total number of ratings in dataset}}{(\text{Total number of users in dataset} \times \text{Total number of items in dataset})}$$

Equation 1: Calculation of Data Density

We plotted this metric and the AUC score of datasets across 14 newly trained (excluding Movies & TV and CDs & Vinyl as they were part of the original paper) Amazon subcategories¹. The analysis showed that smaller datasets, such as Health and Fashion, had lower AUC scores, while larger datasets, such as Books, achieved higher AUC scores. While there is no strong overall correlation between data density and AUC score, in some cases where dataset sizes are relatively comparable such as Software, Industrial, and Video Games, we observe that a dataset with higher data density may be associated with better AUC performance. This suggests that although dataset size plays a more prominent role in model performance, data density can have a localized influence in certain contexts.

¹ All models were trained on 1% of the available data as one of our experiments show that there is marginal AUC score increase when data sample is increased.

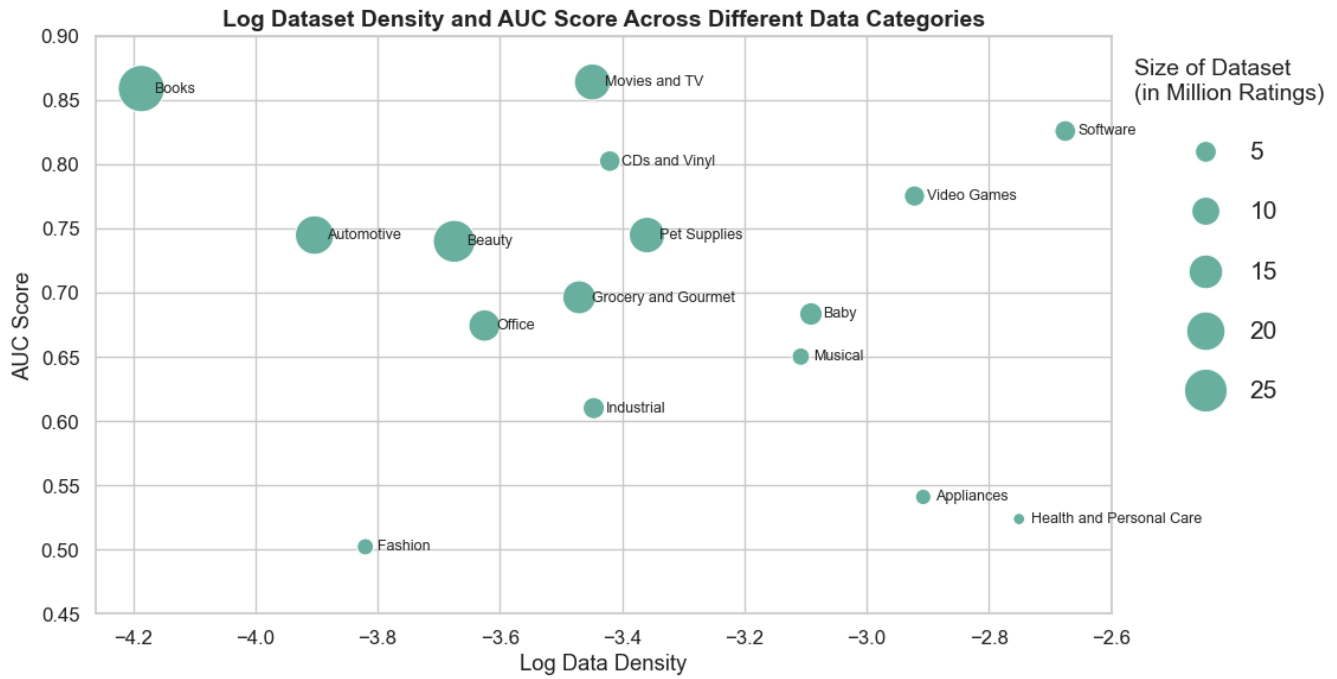


Figure 7. The relationship between data density, data size (number of ratings), and AUC score,

As a summary, the SLi-rec model has poor out-of-the-box transfer performance capacities; thus, training and testing on the same dataset is needed. However, this performance will be highly dependent on the quality of the dataset. Increasing the dataset size or the data density generally leads to better model performance.

Increase diversity in recommendations:

When we add temperature as an inference hyperparameter and normalize the probabilities through softmax, the overall AUC on the movie dataset decreases by 7% (to 0.80), while ordering the probabilities for recommendation by user. Figure 8 shows how the model performs across different levels of temperature and normalized cut-off probability, indicating that a temperature of 0.1 with a prediction threshold of 13% minimizes the losses in the context of 10 options for the user.

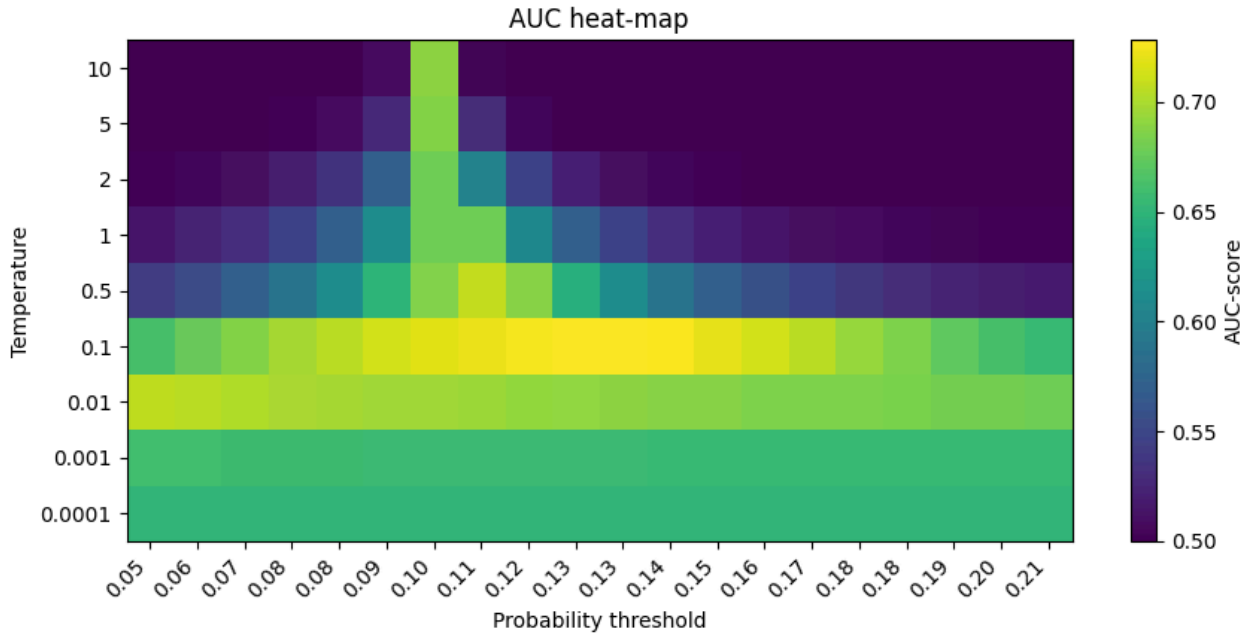


Figure 8: AUC histogram by Temperature and probability threshold. Where the probability threshold defines if the predicted category is 1 or 0.

In this particular context, we can observe how the top probability behaves for a group of 1,000 users. The temperature of 0.1 leads to a Gaussian-looking distribution, being less random than the temperature of 1 and more random than a 0.01 value.

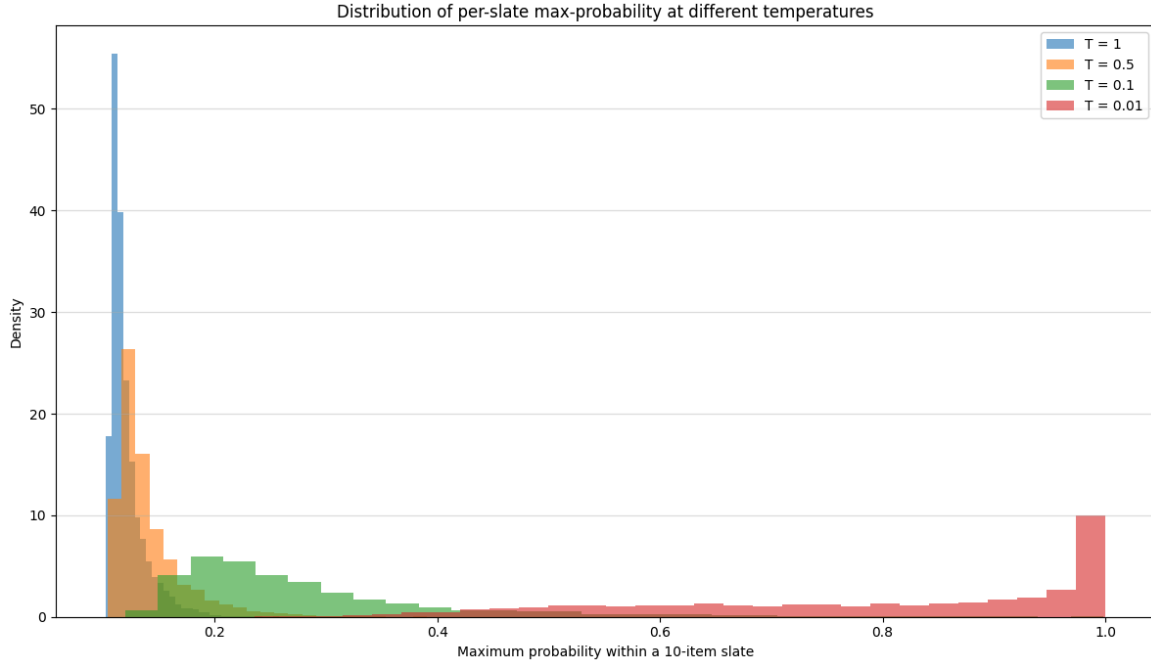


Figure 9: Temperature histogram distribution by temperature level

Figure 10 shows the behaviour across items for a specific user. It is observed that at lower temperatures, the model allocated the majority of its recommendation probability to a small subset of items. In one such case, item 4 received approximately 75% of the likelihood, while items 9,3,5 received the remaining 25%, suggesting that the model strongly favored a narrow set of items when

confident. Conversely, at higher temperature settings, the distribution across items became more uniform, with no clear preference, thus approximating a random guess across the candidate set.

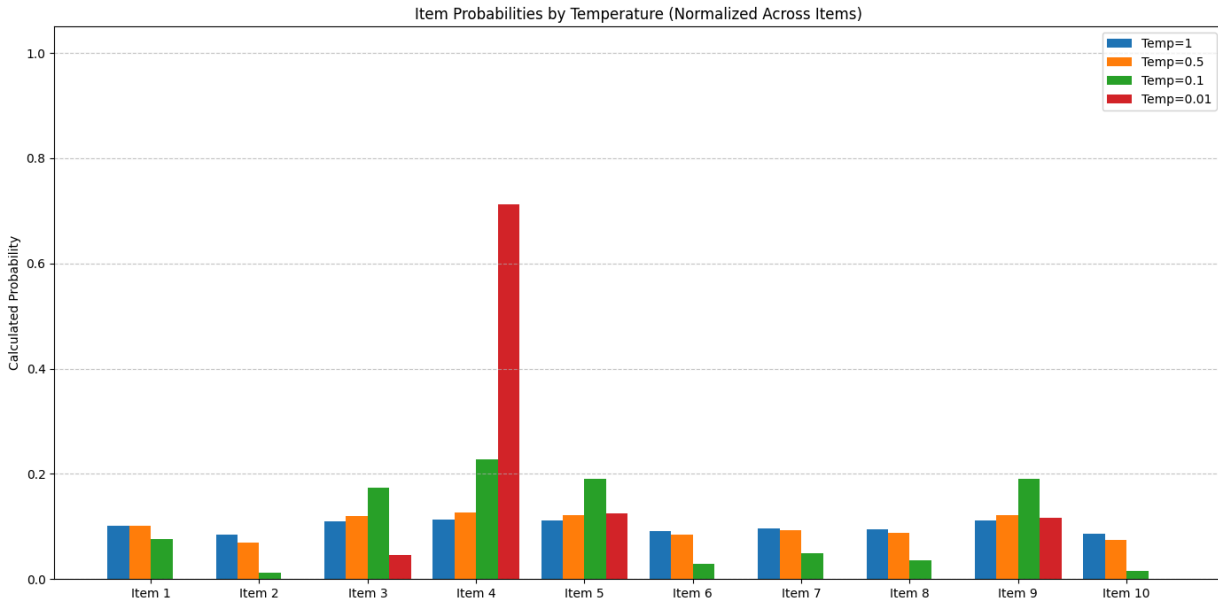


Figure 10: Item probabilities for a Given User under different temperature values

These findings underscore the importance of temperature as a tunable parameter for modulating recommendation determinism and diversity. Its usefulness will vary and will highly depend of the context of applications. In application contexts where a single recommendation must be presented, such as a single personalized search result, lower temperature values may enhance user satisfaction by prioritizing the most relevant items across a set of popular candidates.

Context-Aware SLI-Rec, Fusing Sequential Behavior and Graph-Based Item Relationships:

We evaluated the performance gains achieved by integrating LightGCN-based item embeddings into the SLI-Rec architecture. The goal was to determine whether incorporating structural item relationships could improve top-k recommendation performance, particularly in sparse or cold-start conditions where sequential signals alone may be insufficient.

We compared the hybrid model, SLI_RECWithGCN, against the baseline SLI-Rec using standard ranking and calibration metrics: AUC, MRR, and NDCG@k. Detailed definitions of these metrics are provided in Appendix B.3. Also, learning curves are provided in Appendix B.4 (Figures A1–A2), showing steady improvements in ranking performance and decreasing logloss, indicating stable model convergence.

Table 1. Model Performance Comparison

Metric	Baseline SLI-Rec	Hybrid SLI_RECWithGCN	Brief Description
AUC	0.83	0.8795	Ranking relevant items higher
Mean MRR	0.4946	0.7176	Ranking first relevant item earlier

NDCG@2	0.4075	0.6774	Ranking precision within top-2 items
NDCG@4	0.5107	0.7487	Ranking precision within top-4 items
NDCG@6	0.5607	0.7725	Ranking precision within top-6 items

The above table summarizes the model performance results after 5 epochs of training. The hybrid model consistently outperformed the baseline across all ranking metrics. AUC improved from 0.83 to 0.88, and mean MRR increased from 0.49 to 0.72, indicating that the hybrid model was more effective at surfacing relevant items earlier in the recommendation list. NDCG@6 rose from 0.56 to 0.77, reflecting better quality among the top-6 ranked items. In summary, our results validate that augmenting SLi-Rec with LightGCN-derived item embeddings significantly boosts recommendation performance. The hybrid model consistently outperforms the baseline, delivering stronger top-k precision and better generalization to underrepresented items. By capturing high-order structural relationships from the user-item interaction graph, the model becomes more context-aware and resilient, especially in sparse or cold-start settings.

Ethical Considerations

When building recommendation systems, it's important to think about fairness and the risk of reinforcing bias or limiting what users see. Our project uses anonymized data and includes experiments that aim to make the recommendations more fair and diverse. We also try to clearly explain how the SLi-Rec model works and how our changes help make it easier to understand.

Conclusions

This study demonstrates that targeted enhancements significantly improve the flexibility and robustness of the SLi-Rec model across real-world scenarios. Generalization tests confirmed SLi-Rec's resilience across varying dataset sizes and sampling strategies, provided data representativeness is maintained. Conversely, domain-transfer experiments highlighted its limitations when facing unseen categories, emphasizing the need for retraining. Introducing a temperature hyperparameter effectively balanced recommendation diversity against accuracy, proving beneficial when application context requires adjustable determinism. Most notably, integrating graph-based embeddings with LightGCN substantially enhanced model performance, especially in cold-start settings, illustrating the value of incorporating item relational context. Collectively, these findings advance SLi-Rec as a more adaptive and context-aware recommendation system, optimized for deployment challenges common in practical scenarios.

References.

1. Camilleri, D. (n.d.). Part 2: How much data do I need for a recommendation system? *Shaped Blog*. Retrieved April 16, 2025, from <https://www.shaped.ai/blog/part-2-how-much-data-do-i-need-for-a-recommendation-system>
2. Chen, J., Zhang, H., He, X., Nie, L., Liu, W., & Chua, T. S. (2017, August). Attentive collaborative filtering: Multimedia recommendation with item- and component-level attention. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 335–344).
3. Doms, S., et al. (2016). A framework for dataset benchmarking and its application to a new movie rating dataset. *ACM Transactions on Intelligent Systems and Technology*, 7(3), 1–28. <https://doi.org/10.1145/2751565>
4. Funk, S. (2006, December). Netflix update: Try this at home. Unpublished manuscript.
5. GroupLens Research. (n.d.). MovieLens 100k dataset. Retrieved April 16, 2025, from <https://grouplens.org/datasets/movielens/100k/>
6. Harper, F. M., & Konstan, J. A. (2015). *The MovieLens datasets: History and context*. <https://doi.org/10.1145/2827872>
7. Hidasi, B., & Karatzoglou, A. (2018, October). Recurrent neural networks with top-k gains for session-based recommendations. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management* (pp. 843–852).
8. Hou, Y., Li, J., He, Z., Yan, A., Chen, X., & McAuley, J. (2024). Bridging language and items for retrieval and recommendation [Preprint]. *arXiv*. <https://arxiv.org/abs/2403.03952>
AmazonReviews2023. (2024). *Source code*. GitHub. <https://github.com/hyp1231/AmazonReviews2023>
9. Johnson, M., Schuster, M., Le, Q. V., Krikun, M., Wu, Y., Chen, Z., & Dean, J. (2017). Google's multilingual neural machine translation system: Enabling zero-shot translation. *Transactions of the Association for Computational Linguistics*, 5, 339–351.
10. Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks [Preprint]. *arXiv*. <https://arxiv.org/abs/1609.02907>
11. Koren, Y. (2009, June). Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 447–456).
12. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37.
13. Wang, R., Lou, J., & Jiang, Y. (2023). LightGCAN: A lightweight graph convolutional attention network for user preference modeling and personalized recommendation. *Expert Systems with Applications*, 232, 120741. <https://doi.org/10.1016/j.eswa.2023.120741>
14. Yu, Z., Lian, J., Mahmood, A., Liu, G., & Xie, X. (2019, August). Adaptive user modeling with long and short-term preferences for personalized recommendation. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI-19)* (Vol. 7, pp. 4213–4219). Retrieved from <https://paperswithcode.com/paper/adaptive-user-modeling-with-long-and-short>
15. Zhang, J., Wang, D., & Yu, D. (2021). TLSAN: Time-aware long- and short-term attention network for next-item recommendation. *Neurocomputing*, 441, 179–191. <https://paperswithcode.com/paper/tlsan-time-aware-long-and-short-term-1>

Appendix

Appendix A: Datasets

Table 1. Overview statistics for each Amazon subcategory dataset, with columns detailing the number of users, items, ratings, data density, and log of data density.

Category	Number of Users	Number of Items	Number of Ratings	Data Density	Log of Data Density
Subscription Boxes	15,200	641	16,200	0.1662698087	-0.7791866029
Gift Cards	132,700	1,100	152,400	0.1044050147	-0.981278641
Magazine Subscriptions	60,100	3,400	71,500	0.03499070177	-1.456047347
Software	2,600,000	89,200	4,900,000	0.002112797516	-2.675142122
Digital Music	101,000	70,500	130,400	0.001831332069	-2.737232899
Health and Personal Care	461,700	60,300	494,100	0.001774751971	-2.750862333
Appliances	1,800,000	94,300	2,100,000	0.001237186285	-2.907564903
Video Games	2,800,000	137,200	4,600,000	0.001197417743	-2.921754311
All Beauty	632,000	112,600	701,500	0.0009857623041	-3.006227793
Baby Products	3,400,000	217,700	6,000,000	0.0008106136345	-3.091186096
Musical Instruments	1,800,000	213,600	3,000,000	0.0007802746567	-3.107752499
Handmade Products	586,600	164,700	664,200	0.0006874849787	-3.162736787
Pet Supplies	7,800,000	492,700	16,800,000	0.0004371516448	-3.359367883
CDs and Vinyl	1,800,000	701,700	4,800,000	0.0003800294523	-3.420182744
Industrial and Scientific	3,400,000	427,500	5,200,000	0.0003577571379	-3.446411692
Movies and TV	6,500,000	747,800	17,300,000	0.0003559158146	-3.448652714
Grocery and Gourmet Food	7,000,000	603,200	14,300,000	0.0003386699507	-3.470223335
Kindle Store	5,600,000	1,600,000	25,600,000	0.0002857142857	-3.544068044

Health and Household	12,500,000	797,400	25,600,000	0.0002568347128	-3.590346279
Arts Crafts and Sewing	4,600,000	801,300	9,000,000	0.0002441684437	-3.612310465
Office Products	7,600,000	710,400	12,800,000	0.0002370791844	-3.625106575
Toys and Games	8,100,000	890,700	16,300,000	0.0002259285594	-3.646028867
Patio Lawn and Garden	8,600,000	851,700	16,500,000	0.0002252676589	-3.647301154
Beauty and Personal Care	11,300,000	1,000,000	23,900,000	0.0002115044248	-3.674680543
Amazon Fashion	2,000,000	825,900	2,500,000	0.0001513500424	-3.820017453
Electronics	18,300,000	1,600,000	43,900,000	0.000149931694	-3.824106552
Tools and Home Improvement	12,200,000	1,500,000	27,000,000	0.0001475409836	-3.831087326
Cell Phones and Accessories	11,600,000	1,300,000	20,800,000	0.0001379310345	-3.860338007

Appendix B: Model Architectures and Training Details

B.1 Baseline Model: SLI-Rec

Model Name: `SLI_RECModel`

Architecture Overview: SLI-Rec models both long-term and short-term user preferences by combining an attention-based aggregation of user history with a Time-aware LSTM (Time4LSTMCell). The model constructs a fused user embedding through a learned gating mechanism (alpha fusion) and concatenates it with the target item embedding for final prediction.

- Long-term modeling: Additive attention over historical embeddings (item + category)
- Short-term modeling: Time-aware LSTM on session-level item interactions
- Fusion: A fully connected layer generates a dynamic gating factor (alpha) to blend short-term and long-term embeddings
- Final representation: Concatenation of fused user embedding and target item embedding passed through FC layers

Key Hyperparameters:

- Hidden size: 64

- Attention size: 32
- Embedding dimension (item/user/category): 64
- Dropout: 0.5
- Learning rate: 0.001
- Batch size: 128
- Negative sampling: 1 negative per positive sample
- Epochs: 3
- Loss function: Cross-Entropy

Training Setup:

- Framework: TensorFlow 1.x (graph mode)
- GPU memory growth enabled
- Session-level logging with TensorBoard summaries
- Adam optimizer
- Evaluation metric: AUC, MRR, NDCG@2/4/6, logloss, group AUC

Dataset: Amazon Movies and TV (preprocessed)

- Sequence data created from temporal user-item interactions
- Train/valid/test split based on user history
- Negative sampling performed offline for evaluation

B.2 Proposed Model: SLI_RECWithGCN (Graph-Augmented)

Model Name: **SLI_RECWithGCN**

- Extension Overview: SLI_RECWithGCN enhances the baseline SLI-Rec by fusing static item embeddings generated by LightGCN into the final logit prediction layer. This fusion introduces global structural signals from the user-item interaction graph that are not captured by temporal modeling alone.

- Graph Module:
 - Implemented via LightGCN
 - Learns static embeddings by aggregating over a normalized adjacency matrix of user-item interactions
 - No feature transformation, no non-linearity (as per LightGCN design)
 - 3 layers of propagation (adjacency-based embedding averaging)
- Fusion Strategy:
 - Look up graph-based target item embedding using `tf.nn.embedding_lookup`
 - Concatenate this embedding with the original logit input (user + item + session-level embedding)
 - Feed the fused vector to an FC layer (`logit_fcn_gcn`) for final prediction

LightGCN Hyperparameters:

- Embedding dimension: 64
- Layers: 3
- Learning rate: 0.001
- Batch size: 128
- Loss: BPR loss (matrix factorization + embedding regularization)
- Optimizer: Adam
- Evaluation: MAP, NDCG, Precision, Recall (internally); outputs `ia_embeddings`

Training Pipeline:

- LightGCN model is trained separately on the user-item graph
- Final item embeddings (`ia_embeddings`) are extracted from trained model
- These are injected into SLI-Rec during `SLI_RECWithGCN` instantiation

SLI_RECWithGCN Evaluation Metrics:

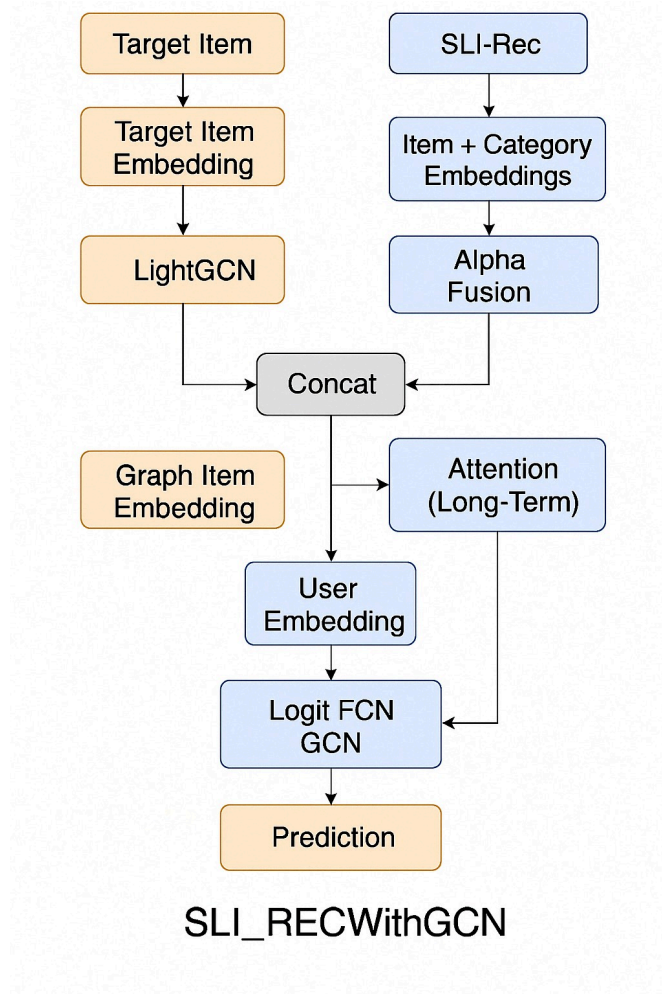
- AUC:0.8795
- MRR: 0.7176
- NDCG@2/4/6: 0.6774 / 0.7487 / 0.7725

Dataset: Same as baseline (10% of Amazon Movies and TV)

- Preprocessing via `amazon_reviews.py`
- Includes vocab generation, timestamp ordering, negative sampling, and session windowing
- Graph construction: bipartite edges from click history

B.3 Architecture Diagram (detailed)

- A visual depiction of SLI_RECWithGCN architecture is provided
- It illustrates the base SLI-Rec components (GRU, attention, gating fusion), LightGCN block, and the fusion at the logit prediction layer



The diagram above shows the full architecture of SLI_RECWithGCN, combining sequential modeling from SLI-Rec with structural signals from LightGCN.

On the left, the target item is embedded and passed through LightGCN, which learns graph-based item embeddings by aggregating over the user-item interaction graph. This captures higher-order relationships, like co-interacted items that don't appear in the same session.

On the right, SLI-Rec models both long-term and short-term user preferences using item and category embeddings. Short-term interests are captured via a GRU (Time4LSTMCell), while long-term behavior is modeled using attention over historical interactions. These are blended using an alpha fusion gate, producing a dynamic user embedding.

The fused user embedding, target item embedding, and graph item embedding are then concatenated and passed through a logit-level FC layer (Logit FCN GCN), which outputs the final recommendation score.

This design allows the model to leverage both session patterns and graph structure, making it more robust—especially for sparse data or cold-start items.

B.3: Evaluation Metrics

We report several standard evaluation metrics to assess the recommendation performance and calibration of our models:

- **AUC (Area Under the ROC Curve):** Measures the ability of the model to rank relevant items higher than irrelevant ones across all users. Higher AUC indicates better overall ranking discrimination.
- **MRR (Mean Reciprocal Rank):** Calculates the average reciprocal rank of the first relevant item in the recommendation list for each user. Higher MRR values indicate that relevant items are recommended earlier.
- **NDCG@k (Normalized Discounted Cumulative Gain at k):** Emphasizes the placement of relevant items within the top-k ranked results. Higher-ranked items are weighted more heavily, rewarding early relevance.
- **Logloss (Logarithmic Loss):** Measures the accuracy of predicted probabilities. Lower logloss values indicate better probability calibration, where predicted scores are closer to actual outcomes.
- **Group AUC:** Computes AUC separately for each user group and averages the results. It captures ranking quality at the individual group level and reduces bias from heavily active users.

B.4: Learning Curves for SLI_RECWithGCN

To better understand the model's training dynamics, we tracked the evolution of key evaluation metrics across epochs during training.

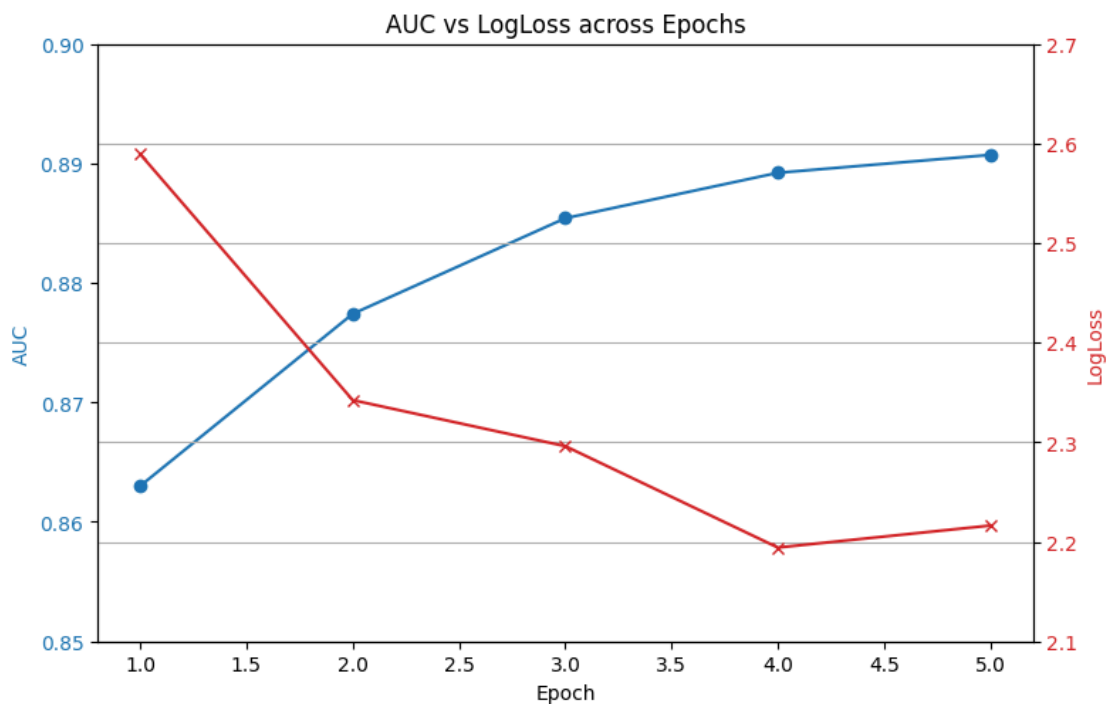


Figure A1. AUC and Logloss progression across training epochs for SLI_RECWithGCN.

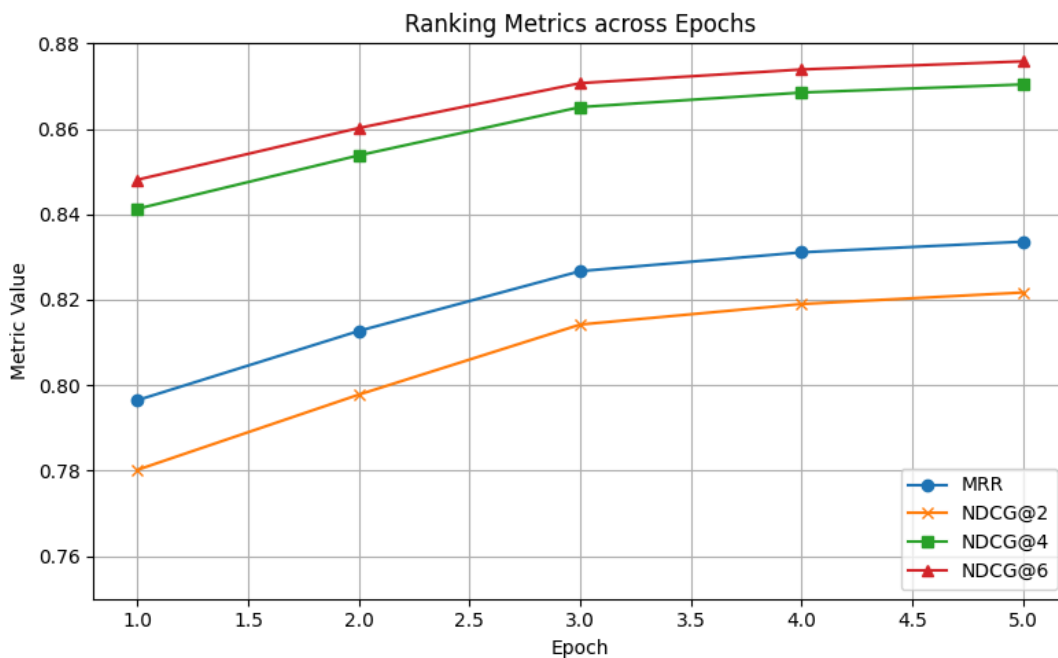


Figure A2. MRR and NDCG@k progression across training epochs for SLI_RECWithGCN.

- Figure A1 shows the progression of AUC and logloss over five epochs. AUC steadily increases, indicating improved ranking discrimination, while logloss generally decreases, reflecting better probability calibration.

- Figure A2 displays the progression of MRR, NDCG@2, NDCG@4, and NDCG@6. These ranking metrics consistently improve across epochs, suggesting that the model became increasingly effective at surfacing relevant items earlier in the recommendation list.

These trends support the stability and effectiveness of the hybrid model's learning process.